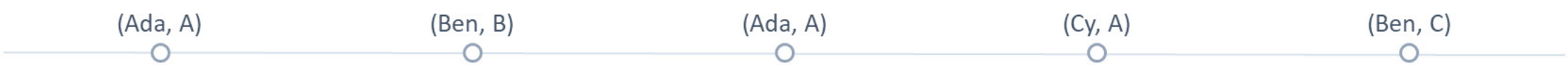
Probabilistic Data Structures

What can we learn from massive data without storing it all?



one stream of events, three different questions

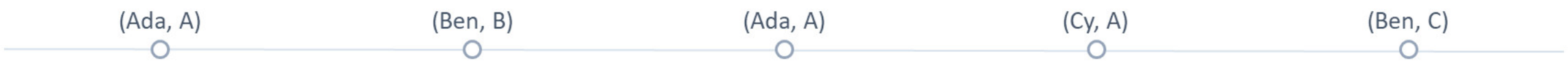
Bloom Filter

Count-Min Sketch

HyperLogLog

Sources S01 Introduction; S02 §1; S03 Introduction.

Three Queries on One Stream



(user, resource) records, in arrival order

Membership Has resource C appeared?

Frequency How many requests target resource A?

Distinct count How many users appeared?

Think for 20 seconds, then answer all three.

Yes C appeared **3** requests target A **3** distinct users

Five records, but only three distinct users.

Repetition adds to a frequency and to the event count. It adds neither a new member nor a new user.

So the same stream needs three different kinds of evidence, and three different error statements.

Sources S02 §2 point queries; S03 Introduction (cardinality). The five records and the answers are a hand-checked teaching example.

The Cost of Exact State

Membership	a set of distinct keys
Frequency	a map from keys to counts
Distinct count	enough state to recognize future duplicates

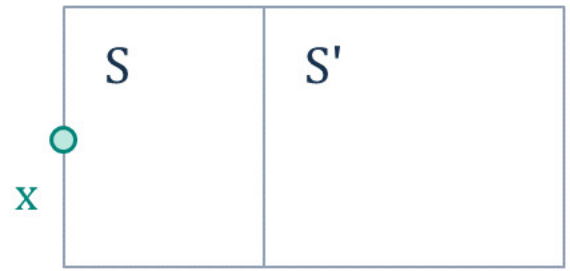
Exact membership over a universe of size u, with n stored keys:

$$b \geq \lceil \log_2 C(u, n) \rceil$$

C(u, n): the number of n-element subsets of a universe of size u.

A bound for exact membership, not for every approximate algorithm.

The distinct count is one number, but a future duplicate still depends on past identities.



Same stored state?

Different exact answers

Two different sets of size n must be told apart, so 2^b states must cover every possible set.

Sources research P1 (counting argument); S01 Introduction; S02 §1; S03 Introduction.

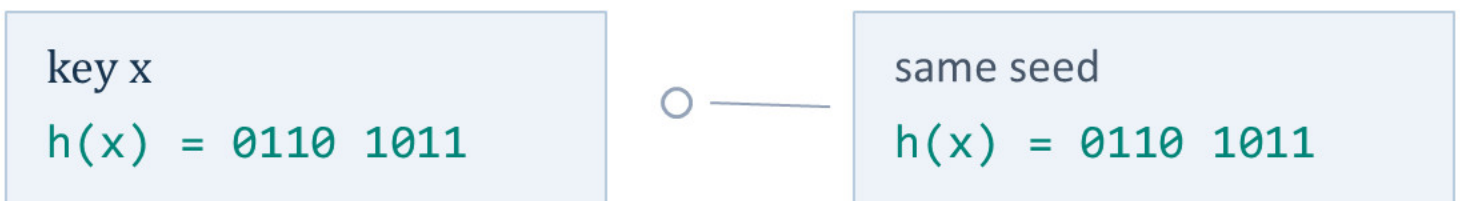
Randomness and Guarantees

Exact identity	true inside the stated model
Approximation	state which step is approximate
High-probability bound	state the failure event and its budget
Asymptotic standard error	a limit scale, not a per-run guarantee

Fixed input. Randomly chosen hash functions. Stable hashes during use.

Draw the randomness once. Re-drawing it per query would break Bloom evidence and HLL de-duplication.

A guarantee for one fixed query is not a guarantee for every query at once.



Stable hashes make Bloom and HLL evidence unchanged by repetition. CMS counts repeated events.

Sources research proof §0 (model vocabulary); S02 §2–3 and Theorem 1; S03 Definition 1 and Theorem 1.

Approximate Membership

$x \in S$?



A positive answer triggers exact verification.

The allowed error is a nonmember that passes the filter and costs extra work.

Rejecting a member is not an allowed error.

Burton H. Bloom, 1970

Sources S01 p. 422 Introduction and A Sample Application; S24 RocksDB Bloom Filter (Life Cycle, Full Filters).

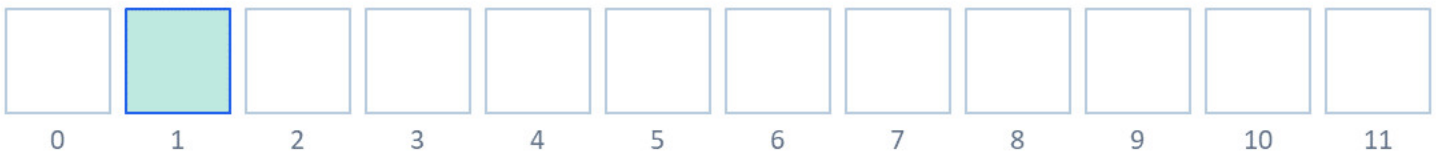
A Bit as Evidence

Store positions, discard keys.

$$B[\,h(x)\,] \leftarrow 1$$

0 no inserted key used this position

1 one or more keys used this position



Illustrative: $a \rightarrow \{1\},\ b \rightarrow \{1\},\ c \rightarrow \{7\}$

a and b share position 1, so a set bit does not say which key put it there.

Position 7 is still 0, so c is rejected even though nothing is stored about c.

One bit gives one piece of evidence.

A zero disproves membership.

A one may be a collision, so it cannot prove membership.

A single probe still accepts many nonmembers.

Next: require several pieces of evidence for each key.

Sources S05 §2.1 and Figure 1, pp. 486–487; research P2. Hash positions are an illustrative assignment.

Bloom Filter Operations

```
insert(x):
    for j = 1..k: B[h_j(x)] = 1
query(x):
    for j = 1..k:
        if B[h_j(x)] == 0:
            return "Definitely absent"
    return "Possibly present"
```

m bits. O(k) update. O(k) worst-case query.

Repeated insertion of one key changes nothing, so n counts distinct keys, not calls.

k = 3 probes per key, m = 12 bits



$a \rightarrow \{1, 4, 9\}$ $b \rightarrow \{4, 7, 10\}$

query c → {1, 4, 10}: all three bits are set

query d → {1, 6, 10}: bit 6 is 0

c is a false positive: nothing was stored for c, yet every probed bit was set by a or b.

d is correctly rejected. The next slide asks what cannot happen.

Illustrative hash assignments, not a measured hash function.

Sources S05 §2.1, pp. 486–488; S01 method analysis; research Bloom §4. Probe positions are illustrative.

Can an Inserted Key Be Rejected?

Insert-only updates. Hashes fixed. No bit is ever cleared.



$a \to \{1, 4, 9\}$ $b \to \{4, 7, 10\}$ shared bit 4

Can an inserted key return “Definitely absent”?

$$x \in S \Longrightarrow B[\, h_j(x)\,] = 1 \quad \text{for all } j$$

No false negatives under insertion-only updates.

Insertion sets every probe bit; later insertions never clear it.

Positive answers may come from collisions, which is the allowed error.

Clearing bit 4 to delete a can make b look absent. Shared bits prevent safe deletion.

Sources research P2 (deterministic invariant); S05 §2.1 and the Counting Bloom discussion.

Expected Bit Occupancy

Model: n distinct keys, k independent uniform probes per key, with replacement.

$$T = kn, \quad q_0 = (1 - 1/m)^T$$

$$X = \text{number of set bits}, \quad EX = m(1 - q_0)$$

Exact probabilities and expectation

One fixed bit



Each probe misses this bit with probability $1 - 1/m$.

All T probes must miss it for the bit to stay 0.

Expectation uses linearity over indicator variables I_b.

Linearity does not require the bits to be independent, and they are not: the T probes are shared.

Sources S05 §2.1, pp. 487–488; research P3. Exact probabilities and expectation under the stated model.

False Positives Conditional on the Array

Fixed nonmember x. Ideal random hashes independent of the stored keys.

$$\Pr(\mathrm{FP} \mid B) = (X / m)^k$$

$$p_{FP} = E[(X / m)^k]$$

$$E[(X / m)^k] \geq (E\,X / m)^k \quad (k \geq 1)$$

Conditional query probability

Average over constructions

Given B with X set bits, one probe succeeds with probability X/m.

Independent query positions give the k-th power. The bits are dependent.



X = 7 set bits, m = 12

All three probes succeed: a false positive.

Replacing X by E X changes the probability.

Jensen gives a lower bound, not an identity.

For k > 1 the exact probability is a random moment, not the moment of the average.

Sources S05 §2.1, pp. 487–490; research P4. S06 is related reading; its exact theorem locator was not confirmed.

A Two-Bit Counterexample

m = 2, n = 1, k = 2

probe positions 00	probe positions 01	probe positions 10	probe positions 11
			
X = 1	X = 2	X = 2	X = 1

Four equally likely probe sequences; two occupy one bit, two occupy both.

$$\Pr(X = 1) = \Pr(X = 2) = 1/2$$

$$p_{FP} = (1/2)(1/2)^2 + (1/2)(1)^2 = 5/8$$

$$(E X / m)^2 = (3/4)^2 = 9/16$$

5/8 ≠ 9/16

Exact enumeration, not an experiment.

The gap is the construction variance that a mean-occupancy shortcut throws away.

Sources research P4 exact enumeration; S06 is thematically related but this example is not claimed to be its original counterexample.

Two Approximation Steps

$$\mathrm{E}[\,(X/m)^k\,] \approx [\,1-(1-1/m)^{kn}\,]^k$$

Step 1: concentrated occupancy

$$\approx (\,1-e^{-kn/m}\,)^k$$

Step 2: exponential approximation

Fixed k and kn = Θ(m) give a useful large-filter regime.

Step 1 replaces the random occupancy ratio by its mean, justified by concentration with an explicit failure budget η.

Step 2 uses $\ln(1 - 1/m) = -1/m + O(m^{-2})$.

Neither step is “the bits are independent”.

The objective optimized next is the last line, so its optimum is a design rule, not the exact optimum for every finite filter.

Sources S05 §2.1, pp. 487–490; research P5. The concentration step cites a standard inequality without reproving it.

Choosing the Probe Count

$$a = n / m, \quad F(k) = (1 - e^{-ak})^k$$

$$\ln F(k) = k \cdot \ln(1 - e^{-ak})$$

$$d/dk \ \ln F(k) = \ln(1 - e^{-ak}) + ak\, e^{-ak} / (1 - e^{-ak})$$

Substitute ak = ln 2: the two terms cancel.

$$k^* = (m / n) \ln 2 \quad \text{(stationary point)}$$

Two competing effects

more probes → stricter rejection

more probes → fuller array

Optimum: about half the bits are set.

Next: prove global optimality using the derivative sign.

Then choose integer k and check CPU cost.

Sources S05 §2.1, p. 489; research P6. Continuous optimization of an approximate objective.

The Half-Occupancy Optimum

$$z = e^{-ak}, \quad \ln F = -a^{-1} \ln z \cdot \ln(1 - z)$$

z: approximate empty fraction

$$D(z) = z \ln z - (1 - z) \ln(1 - z)$$

$$D''(z) = 1/z - 1/(1 - z) > 0 \quad (0 < z < 1/2)$$

D(0) = 0 as a limit and D(1/2) = 0, so strict convexity forces D < 0 inside; therefore G′ > 0 on (0, 1/2) and, by symmetry, G′ < 0 on (1/2, 1).

$$k^* = (m / n) \ln 2, \quad F^* = e^{-(m/n)(\ln 2)^2}$$

Unique continuous minimum of F at half occupancy.

Where the pieces come from

$$G = \ln z \cdot \ln(1 - z)$$

$$z(1 - z)\, G' = -D(z)$$

$$z = 1/2 \Leftrightarrow ak = \ln 2$$

Strict convexity gives D < 0 on the left half. Symmetry reverses the derivative sign on the right.

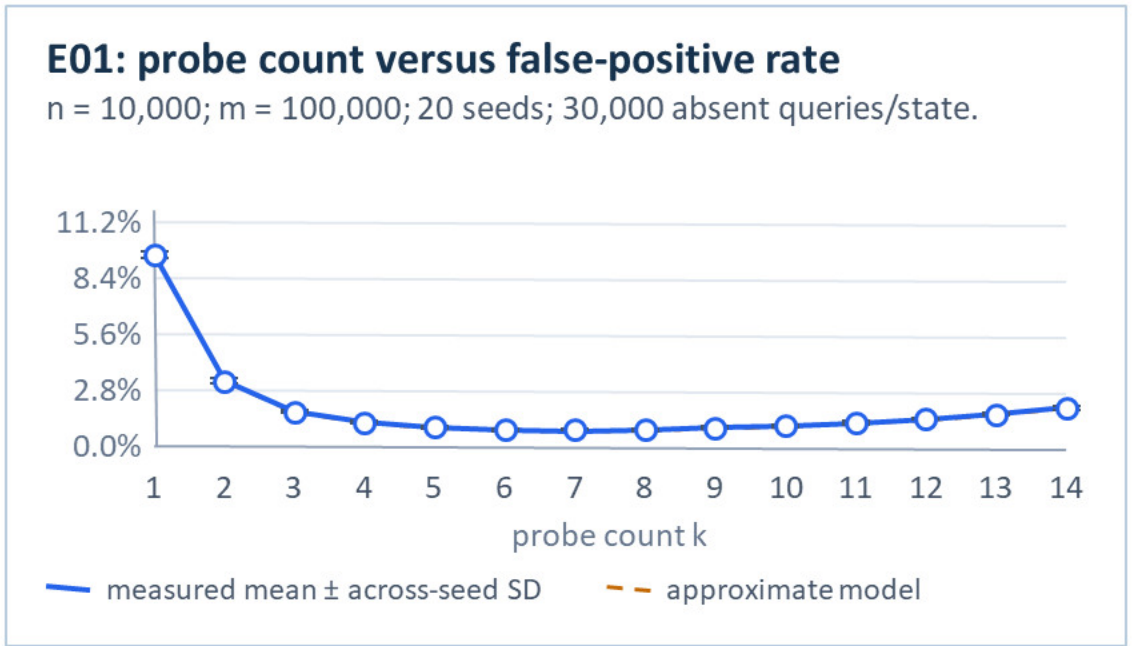
Sources S05 §2.1, p. 489; research P6 full derivative-sign argument. Global optimum inside the approximate model.

A Bloom Space Budget and Its Test

$$m \approx -\, n \ln p_{FP} \,/\, (\ln 2)^2 , \quad k \approx \log_2(1 / p_{FP})$$

n = 10^6, target p_FP = 0.01: about 9.59 bits/key, k = 7

Logical payload only; metadata and allocator overhead are separate.



Theoretical prediction: the approximate model.

Measured: E01, an offline run of this deck.

The minimum of the measured mean is at k = 7, close to the continuous optimum (m/n) ln 2 ≈ 6.93.

Error bars are across-construction standard deviations, not query confidence intervals.

The 240-million-query extension was not run.

Sources S05 §2.1 parameter analysis; research Bloom §6; E01 measured data from experiments/results/lecture/E01_summary.csv and E01_trials.csv.

Bloom Filters in the Lookup Path

$$E\,C = c_B + [\,\pi + (\,1 - \pi\,)\,p_{FP}\,]\,c_E$$

π: lookup hit rate c_B: filter cost c_E: exact lookup cost

Favourable when the avoided exact lookups outweigh the filter cost.

Hits always reach the exact store; misses reach it only through a false positive.

The full condition $c_B < (1 - \pi)(1 - p_{FP})\,c_E$ stays in the speaker notes.

Next question: how many times, not whether.

RocksDB: filters before SST lookups

A negative answer skips the file lookup.

Full and partitioned layouts trade locality against occupancy spread.

Limits to keep in view

Shared bits prevent direct deletion.

Double hashing and blocked layouts change the analysis.

A positive answer is never the final word.

Sources S24 RocksDB Bloom Filter (Life Cycle, Full Filters, The math); S07 Abstract and §1; research applications §2.2 cost model.

Frequency Queries on a Stream

Updates change a frequency vector

(A,+1) (B,+1) (A,+1) (A,+1) (C,+1)

$f_x \leftarrow f_x + \Delta; \quad F_1 = \Sigma_y f_y$

Exact: store every key and its count.

Sketch: store w × d counters and no key identities.

A named-key estimate uses error units tied to the total mass F_1.

Cormode & Muthukrishnan (2005), S02 §1–2. Fixed-key frequency queries.

Collision Noise in One Row

One bucket combines contributions from several keys.

Target x	Colliding keys	Bucket reading
3	**5**	**8**

$$C_{j,h_j(x)} = f_x + Z_j$$

$$Z_j = \Sigma_{y \neq x} f_y \cdot 1[h_j(y) = h_j(x)] \geq 0$$

A second row sees different collisions. How should we combine readings?

S02 §3–4.1; research P7. Exact state identity; illustrative collision.

How Should We Combine Independent Rows?

Three row readings for one key

Row 1	**8**
Row 2	**4**
Row 3	**3**

Min, mean, or max?

Decide before you click.

All three readings describe the same key.

Think for 15 seconds: min, mean, or max?

Minimum = 3

```
update(x, delta):
    for j = 1..d: C[j, h_j(x)] += delta
query(x): return min(C[j, h_j(x)] for j = 1..d)
```

Why the minimum

Every row reading is the true frequency plus nonnegative collision noise.

Nonnegative frequencies imply nonnegative collision noise.

The minimum keeps the true mass and keeps the smallest noise, not the largest.

A nonnegative counter array alone does not prove $f \geq 0$; the caller must keep the true current vector nonnegative.

Source S02: Cormode and Muthukrishnan (2005), sections 3 and 4.1. Values 8, 4, 3 are a teaching construction.

The Count-Min Point-Query Guarantee

Assumptions

Fixed $f \geq 0$ and x, chosen before the hashes. Faithful counters, no overflow.

Pairwise collision probability $\leq 1/w$ per row. Independent rows.

$$w = \lceil e/\varepsilon \rceil; \quad d = \lceil \ln(1/\delta) \rceil \quad (0 < \varepsilon, \delta < 1)$$

$$f_x \leq \hat{f}_x \quad \text{always}$$

$$\Pr(\hat{f}_x > f_x + \varepsilon F_1) \leq \delta$$

One-sided additive error for a fixed point query.

S02 §4.1 Theorem 1 (author PDF pp. 6–7). Fixed f and fixed query x.

Proof Step 1: Isolating the Noise

$$C_{j,h_j(x)} = f_x + Z_j$$

$$Z_j = \Sigma_{y \neq x} f_y \cdot 1[h_j(y) = h_j(x)]$$

$$f \geq 0 \text{ implies } Z_j \geq 0$$

Every row reading is an upper estimate. Their minimum is still an upper estimate.

Nonnegative bucket totals alone do not establish nonnegative frequencies.

S02 §4.1; research P7 step 1. Deterministic, before probability.

Proof Step 2: Expected Collision Mass

$$E[Z_j] = \Sigma_{y \neq x} f_y \cdot \Pr(h_j(y) = h_j(x))$$

$$\leq (F_1 - f_x) / w \leq F_1/w$$

Only the pairwise collision bound is used here.

Linearity of expectation does not require independent collision indicators.

Seeded BLAKE2b is reproducible engineering; these ideal assumptions remain explicit.

S02 §4.1; research P7 step 2. Pairwise collision bound and linearity.

Proof Step 3: A Good Row

$$Z_j \geq 0,\ t > 0: \quad \Pr(Z_j > t) \leq E[Z_j]/t$$

$$t = \varepsilon F_1: \quad \Pr(Z_j > \varepsilon F_1) \leq 1/(w\varepsilon)$$

$$w \geq e/\varepsilon \ \text{ implies } \ \Pr(\text{row j is bad}) \leq 1/e$$

$F_1 = 0$ is separate: every counter and estimate is exactly zero.

A bound on failure probability is not a prediction of measured error.

S02 §4.1; research P7 step 3. Markov for nonnegative noise.

Proof Step 4: All Rows Must Fail

The minimum fails only when every row fails.

$$\Pr(\hat{f}_x > f_x + \varepsilon F_1) = \prod_{j=1}^{d} \Pr(Z_j > \varepsilon F_1)$$

$$\leq (1/e)^d = e^{-d} \leq \delta$$

Independent row hashes give independent bad events for fixed f and x.

Copying one row keeps the same failure probability p, where $p \leq 1/e$.

Copies do not turn p into p^d.

S02 §4.1; research P7 step 4. Independence between rows.

Error Scale and Space

$$\hat{f}_x - f_x \le \varepsilon F_1 \quad \text{with probability} \ge 1 - \delta$$

If $f_x \ge \phi F_1$, the relative error is at most ε/ϕ on that event.

Example: $\varepsilon = 10^{-3}$, $\delta = 10^{-6}$

w = 2719, d = 14: 38,066 counters

64-bit counters: 304,528 bytes ≈ 297.4 KiB

wd counts counters, not bits. Rare-key relative error can still be large.

S02 §4.1; research CMS §7. Design arithmetic, not a measurement.

Update Models and Query Quantifiers

Insertion-only	$\Delta \geq 0$	Nonnegative f
Strict turnstile	Negative updates allowed	Every $f_v \geq 0$ at all times
General turnstile	Signed final coordinates	The one-sided min proof fails

Fixed query: failure $\leq \delta$.

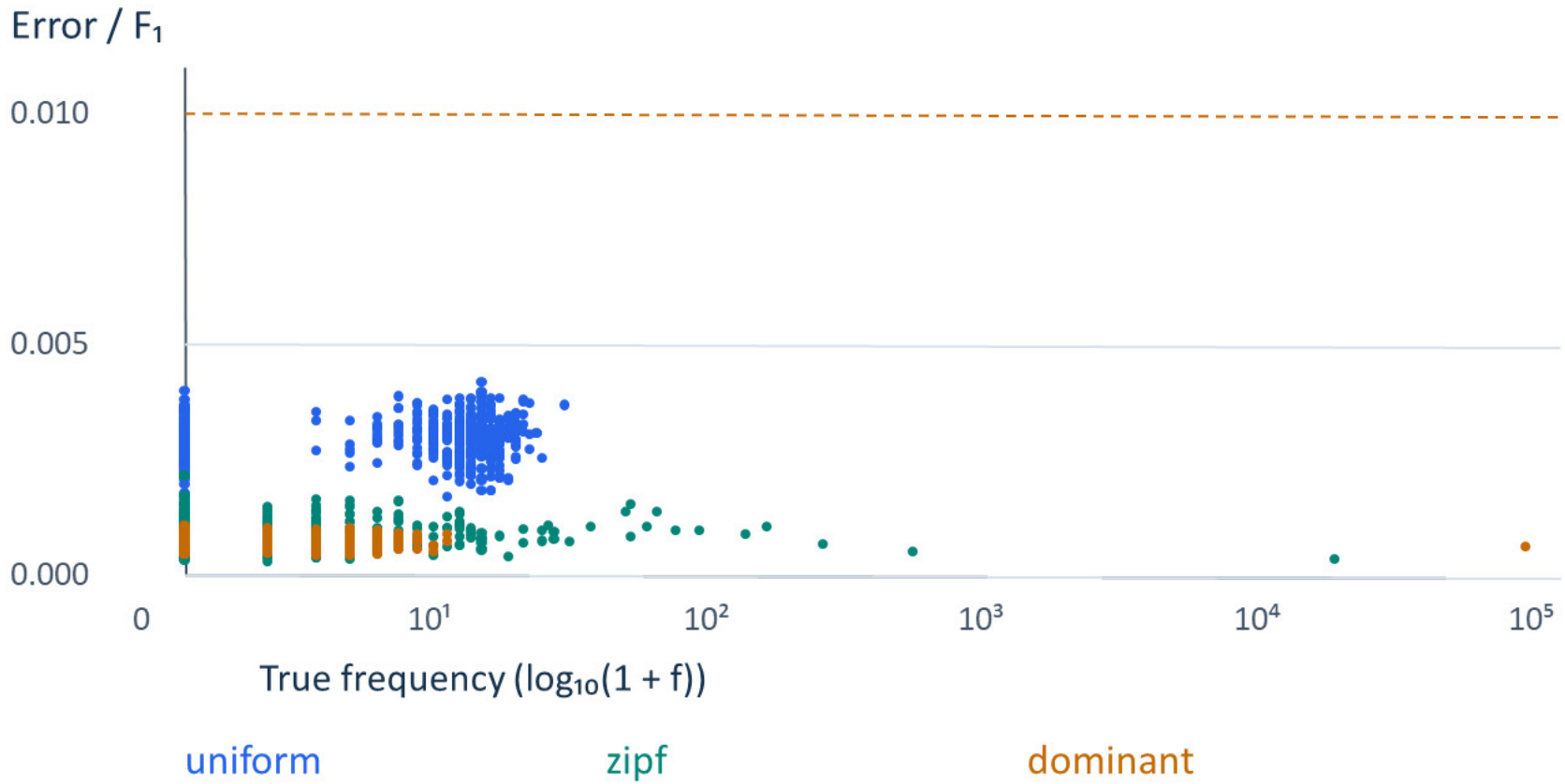
q predetermined queries: union bound $\leq q\delta$. Total budget Δ uses $\delta = \Delta/q$.

Queries chosen after seeing the sketch require separate analysis.

S02 §2, §4.1 Theorems 1–2; research P8/P9. Models and quantifiers.

Testing and Merging Frequency Summaries

E04: 20 seeds × 500 queries × 3 distributions. Plot: seed 1001.



30,000 queries

No negative errors

Max error/F_1

0.00426

Compatible matrices add exactly. Replayed events count again.

Measured E04_queries.csv; S02 §4.1, §5. Ordinary linear CMS merges by addition.

Distinct Users, Repeated Events

Five events

Ada Ben Ada Cy Ben

Three distinct identities

An exact set must recognize every future duplicate.

A small summary needs a signal that repetition cannot increase.

Idea: give each identity a stable random-looking bit string.

Rare Prefixes Reveal Scale

Rank ρ: position of the first 1, starting at position 1

0 0 0 1 …

What is its rank? Think for 15 seconds.

ρ = 4

$$\Pr(\rho > r) = 2^{-r} \quad (r \geq 0)$$

The first r independent fair bits must all be zero.

S03 §1 and Figure 2. Ideal independent fair-bit suffixes.

The Distribution of a Maximum

s distinct keys, independent ideal suffixes; $s \geq 1$, integer $r \geq 0$

$$M = \max(\rho_1, \ldots, \rho_s)$$

$$\Pr(M \leq r) = (1 - 2^{-r})^s$$

All s ranks must be at most r. Independence gives the product.

$$s \cdot 2^{-r} \approx 1 \quad \text{implies} \quad r \approx \log_2 s$$

Exact CDF; approximate scale. One maximum remains noisy.

2^M alone is neither an exact count nor an unbiased estimator.

S03 §2; research P11. Exact ideal-model CDF, approximate scale.

One Hash, Many Registers

Hash prefix selects a bucket. Suffix supplies a rank.

1010

prefix: p = 4 bits, bucket 10

0001…

suffix: ρ = 4

$$m = 2^p \qquad M[j] \leftarrow \max(M[j], \rho(w))$$

State in bucket 10

0	4	4	4	6

Incoming ranks: 4, 2, 4, 6

A repeated identity has the same hash and cannot change a maximum.

S03 Figures 2–3. Illustrative hash assignments.

Three HLL Models

Model	Empty value	Hash model	Role
Original theory	$-\infty$	Infinite suffix	Figure 2 / Theorem 1
Teaching model	0	Infinite suffix	Couple to the theory
Implementation	0	L-bit hash	Cap the all-zero suffix

$$q_H = L - p_H; \quad \text{all-zero suffix: } \rho = q_H + 1$$

The finite estimator does not inherit every infinite-model theorem.

S03 Figure 2, Theorem 1 and §4(i); S04 §5.

A Register Distribution and Empty Buckets

Fix bucket j and integer threshold $r \geq 0$.

$$\Pr(\text{key chooses } j \text{ and } \rho > r) = 1 / (m \cdot 2^r)$$

$$\Pr(M_j \leq r) = (1 - 1 / (m \cdot 2^r))^n$$

Every distinct key must avoid that bucket-threshold event.

$$\Pr(\text{any empty bucket}) \leq m(1 - 1/m)^n \leq m \exp(-n/m)$$

Couple the two infinite models using the same hashes.

They agree when every bucket is nonempty. Fixed-m moment differences vanish.

Finite suffixes: use the CDF for $0 \leq r \leq q$; for $r \geq q+1$, CDF = 1 ($q = L-p$).

S03 §2; research P11/P13. Fixed n, independent distinct-key hashes.

The Harmonic Estimator

Many maxima give many coarse bucket-size signals.

$$E_0 = \alpha_m \, m^2 \, / \, [\, \Sigma_{j=1}^{\,m} \, 2^{-M_j} \,]$$

Harmonic aggregation reduces the influence of unusually large signals.

α_m calibrates the estimate. Its value requires analysis.

E[1/S] generally differs from 1/E[S].

At n = 0, zero-initialized raw = $\alpha_m m$. Small-range correction comes next.

S03 §2, Figure 2; S04 Figure 1. Calibration requires analysis.

Asymptotic Standard Error

Quoted asymptotic theorem

$$E[E_{th}]/n = 1 + \delta_1(n) + o(1)$$

$$SD(E_{th})/n = \beta_m/\sqrt{m} + \delta_2(n) + o(1)$$

$$\beta_m \to \sqrt{(3 \ln 2 - 1)} \approx 1.03896$$

Figure 2: empty = $-\infty$, infinite ideal hashes, fixed $m \geq 16$, then $n \to \infty$.

δ_1 and δ_2 are oscillatory corrections. RSE $\approx 1.04/\sqrt{m}$ is a standard deviation scale.

E08, m = 4096: three different objects

Measured SD 1.420%; seed-bootstrap 95% interval [1.281, 1.553]%.

Quoted asymptotic reference 1.625% lies outside this empirical interval.

S03 Figure 2, Theorem 1 (PDF p. 5); E08_summary.csv.

Small Cardinalities and Finite Hashes

Use the empty buckets as a second scale signal.

$$V = \#\{\, j : M_j^{(0)} = 0 \,\}$$

$$E[V] = m(1 - 1/m)^n \approx m \exp(-n/m)$$

$$\hat{n}_{LC} = m \ln(m/V) \quad (V > 0)$$

This is a plug-in estimate, not an exact inversion of random V.

At n = 0: theoretical raw = 0; zero raw = $\alpha_m m$; Linear Counting = 0.

Finite hashes need a rank cap. HLL++ adds sparse storage and empirical bias correction.

S03 Figure 3 and §4; Heule et al. (2013), S04 §5.

Combining Two HLL States

Illustrative compatible states

A

2	0	3	1

B

1	4	2	0

How should they combine? Think for 15 seconds.

$$M_j(A \cup B) = \max(M_j(A), M_j(B))$$

Illustrative answer: [2, 4, 3, 1]

A	7	8	5	8	7	4	7	8	10	4	8	5	3	5	4	4
B	5	7	4	4	3	4	4	5	6	9	3	4	4	4	3	8
merged	7	8	5	8	7	4	7	8	10	9	8	5	4	5	4	8
central	7	8	5	8	7	4	7	8	10	9	8	5	4	5	4	8

Measured E07: merged = central; repeating every key 10 times changes no register.

S03 §4; measured E07_state.json. Compatible hashes and precision.

HLL Space, Validation, and Aggregation

Space and interfaces

$m \approx (1.04/s)^2$ for target typical RSE s

m = 16,384: typical RSE ≈ 0.8125%

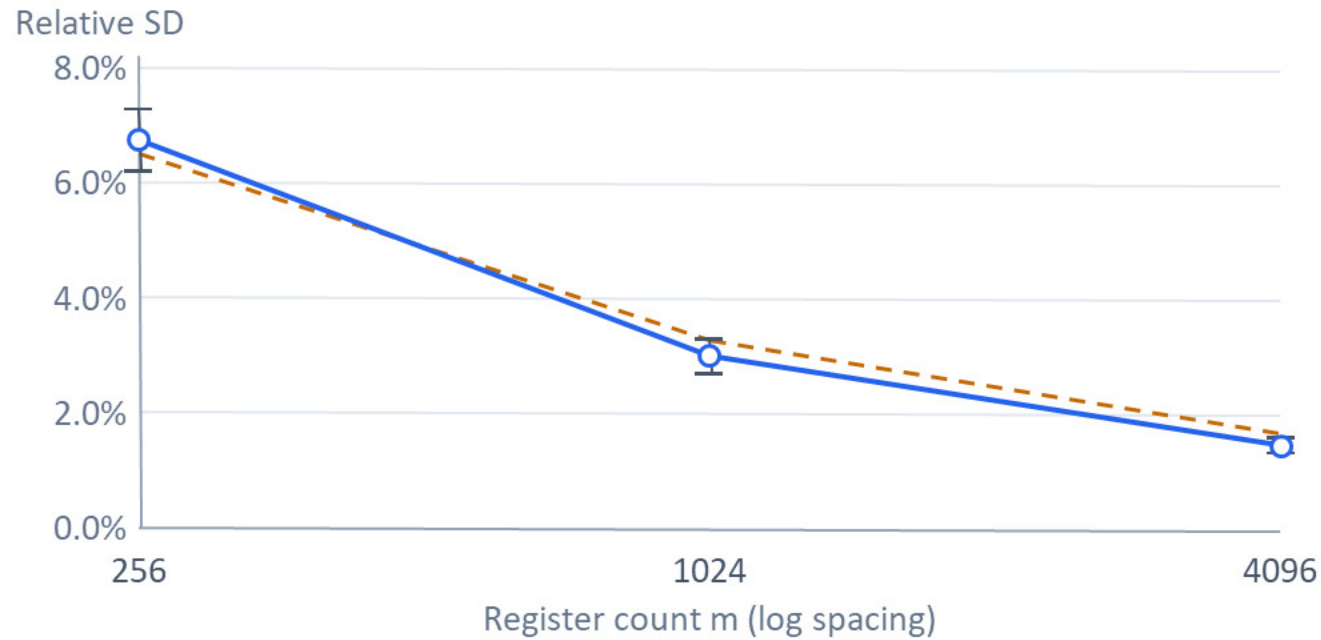
64-bit hash, p = 14: 6-bit registers

Logical payload = 12 KiB

Update O(1); estimate and merge O(m)

BigQuery: store sketches, then merge

State maxima merge exactly; counts remain estimates.



m = 4096: SD 1.420%; bootstrap [1.281, 1.553]%; reference 1.625%.
This interval excludes the reference. The run does not prove the asymptotic constant.

S03 Theorem 1; S04 §5; BigQuery HLL_COUNT (S26); measured E08.

Cuckoo Filter: Recoverable Alternatives

A fingerprint may occupy either of two recoverable buckets.

Bucket 3

t

Bucket 5

t

Alternative homes: one stored copy

$$3 \text{ XOR } 6 = 5; \quad 5 \text{ XOR } 6 = 3$$

$$(i \text{ XOR } g(t)) \text{ XOR } g(t) = i$$

Delete only known-present copies. Preserve a displaced victim on insertion failure.

Fan et al. (2014), S08 §3–4. Illustrative bucket indices.

Which Summary Fits the Query?

Avoid disk reads for missing keys.

Estimate the request count of a named key.

Count unique users across shards.

Choose the query, the error model and the update model.

Summary	Query	Error	Updates	Merge
Bloom	Membership	False positives	Insert-only	Bitwise OR
Count-Min	Frequency	$+\varepsilon F_1$ with $1-\delta$	$f \geq 0$	Counter sums
HLL	Distinct count	Typical SD scale	Insert-only	Register maxima

The guarantee needs compatible hashes, valid updates and the correct quantifier.

S01/S02/S03/S08. Compare query, update model and error semantics.

What Information Can We Afford to Forget?

The query determines what to remember.

Compression changes the kind of error we accept.

A guarantee is meaningful only with its assumptions.

Query Information kept Error model Assumptions

Bloom (1970); Cormode & Muthukrishnan (2005); Flajolet et al. (2007).

Questions and Discussion

Which assumptions would your application violate?

What extra information would deletion or Top-K require?

10 minutes; appendix slides are optional.

Poissonization and the HLL Proof Boundary

Poisson total N ~ Poisson(n): independent bucket occupancies

$$\lambda = n/m; \quad \Pr(M_j^{(0)} \le r) = \exp(-\lambda \cdot 2^{-r})$$

$$Y_0 = 2^{-R}; \quad E[Y_0^t] = \Sigma_{r \ge 0}\, 2^{-tr} \Pr(R = r)$$

$$R = M^{(0)}, \quad t > 0, \quad \text{integer } r \ge 0$$

Zero initialization gives finite inverse-rank moments.

Figure 2: an empty $-\infty$ register makes 2^{-M} infinite.

Mellin transforms, reciprocal moments and de-Poissonization remain cited.

S03 §2–3; research P12/P13. Teaching derivation, not a full Mellin proof.

MinHash and Jaccard Similarity

Nonempty fixed sets; uniformly random minwise permutations.

$$J(A,B) = |A \cap B| \,/\, |A \cup B|$$

$$\Pr(\min \pi(A) = \min \pi(B)) = J(A,B)$$

The smallest element of the union lies in the intersection exactly on a match.

$$\hat{J} = (1/k) \sum_{i=1}^{k} 1[\, \min \pi_i(A) = \min \pi_i(B) \,]$$

Independent minwise permutations: unbiased, variance $J(1-J)/k$.

Define empty-set cases explicitly; practical hashing needs its own assumptions.

Broder (1997), S09; research P16. Independent minwise permutations.

Candidate Keys and Tail Quantiles

Top-K needs candidate identities

CMS answers a named key. Misra–Gries / Space-Saving retain candidates.

Tail quantiles need order information

t-digest: practical tail accuracy, estimator/version-dependent behavior.

KLL: probabilistic rank-error guarantees under a stated model.

Membership, frequency and cardinality summaries cannot answer every query.

S10 Misra–Gries; S11 Space-Saving; S12 Count Sketch; S15 t-digest; S16 KLL.